

# Projet | Conception

Andreea-Monica Balan | Ivan Hegedić

✉ [andreea-monica.balan@epfl.ch](mailto:andreea-monica.balan@epfl.ch) | ✉ [ivan.hegedic@epfl.ch](mailto:ivan.hegedic@epfl.ch)

## Table des matières

|    |                                                                               |    |
|----|-------------------------------------------------------------------------------|----|
| 1  | Hiérarchie générale des classes . . . . .                                     | 2  |
| 2  | Classe Vecteur . . . . .                                                      | 4  |
| 3  | Classe ObjetMobile . . . . .                                                  | 5  |
| 4  | Classe ObjetPhysique . . . . .                                                | 5  |
| 5  | Classe PointMateriel . . . . .                                                | 6  |
| 6  | Classe PointCharge . . . . .                                                  | 6  |
| 7  | Classes ChampForces et ChampComposee . . . . .                                | 7  |
| 8  | Classe ForceCentrale . . . . .                                                | 7  |
| 9  | Classe ChampNewtonien . . . . .                                               | 8  |
| 10 | Classe GravitationConstante . . . . .                                         | 8  |
| 11 | Classes ChampElectrique, ChampElectriqueConstant et ChampMagnetique . . . . . | 9  |
| 12 | Classe ChampRestreint . . . . .                                               | 9  |
| 13 | Classes Contrainte et ContrainteComposee . . . . .                            | 10 |
| 14 | Classes Collision, Elastique et Plastique . . . . .                           | 11 |
| 15 | Classe PlanImmobile . . . . .                                                 | 12 |
| 16 | Classe ContrainteSpherique . . . . .                                          | 12 |
| 17 | Classes EulerCromer, Newmark et RungeKutta . . . . .                          | 13 |
| 18 | Classe Systeme . . . . .                                                      | 13 |
| 19 | Affichage graphique avec Qt5 . . . . .                                        | 14 |

# 1 Hiérarchie générale des classes

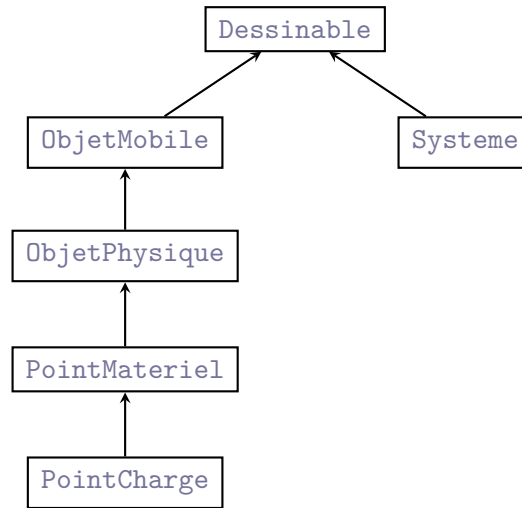


FIGURE 1 – Hiérarchie dans l'arbre de `Dessinable`.

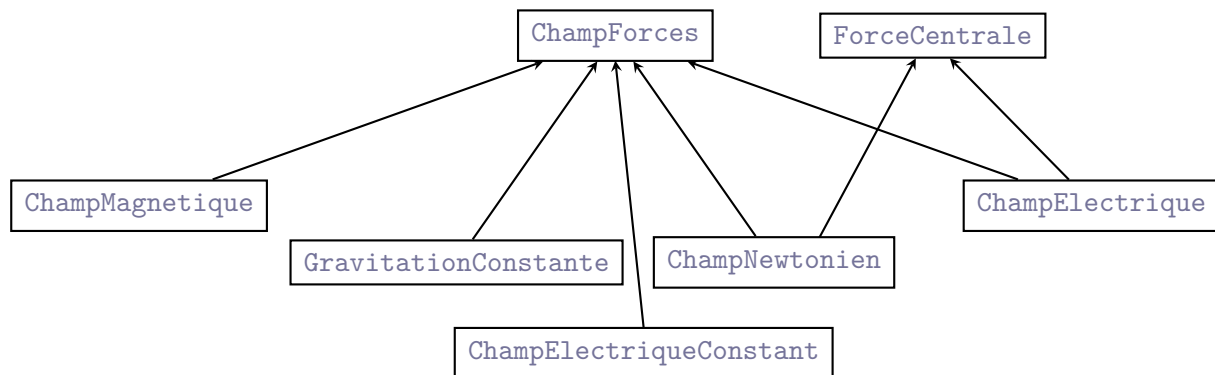
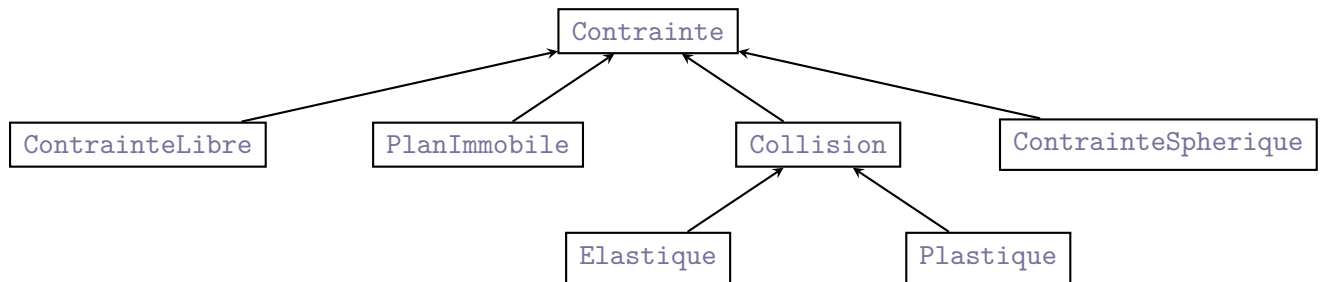
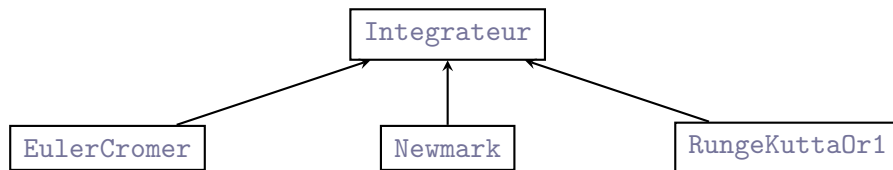
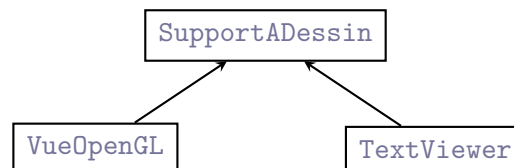


FIGURE 2 – Hiérarchie dans l'arbre de `ChampForces` et de `ForceCentrale`.

FIGURE 3 – Hiérarchie dans l'arbre de `Contrainte`.FIGURE 4 – Hiérarchie dans l'arbre de `Integrateur`.FIGURE 5 – Hiérarchie dans l'arbre de `SupportADessin`.

## 2 Classe Vecteur

L'unique attribut de la classe `Vecteur` est un `std::vector<double>` et la classe dispose des méthodes (dont quelques-unes) affichées dans le diagramme ci-dessous et celles plus compliquées expliquées plus bas :

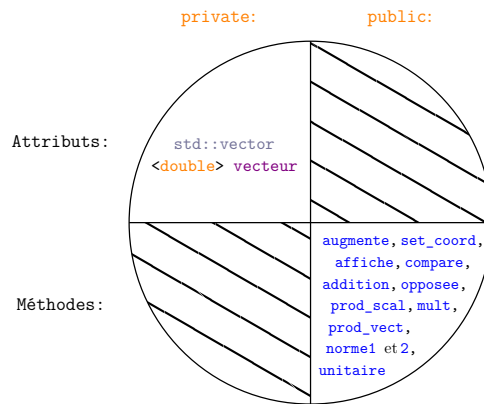


FIGURE 6 – Conception de la classe `Vecteur`.

`Vecteur addition(const Vecteur& v) const` sert à additionner deux `vecteurs` coordonnée par coordonnée. Le processus est fait tout d'abord en vérifiant si les deux `vecteurs` sont de même dimension – si oui, alors la méthode procède par une boucle qui additionne les coordonnées appropriées – si non, on appelle la méthode `plongement` en ajoutant la valeur zéro certain nombre de fois adéquat jusque ce que les dimensions des `vecteurs` ne soient égales, puis on additionne leurs coordonnées.

`double prod_scal(const Vecteur& v) const` effectue le produit scalaire de deux `vecteurs`. Selon notre convention choisie, si les deux ne sont pas de même dimension, alors l'un d'entre eux devrait être fait plonger dans l'espace vectoriel de plus grande dimension. Puisque les nouvelles coordonnées attribuées à celui-là sont toutes zéro, alors la boucle dans cette méthode n'effectue les multiplications successives des paires de coordonnées appropriées que jusque ce que la dimension minimale des deux `vecteurs` ne soit atteinte. Cette dimension-là est trouvée par la méthode `getDimMin`.

`Vecteur unitaire() const` calcule le vecteur unitaire du `vecteur` sur lequel la méthode opère en le divisant par sa norme calculée par `norme1`. Au cas où la méthode opèrerait sur le vecteur nul (ou un autre `vecteur` dont la norme est plus petite que certaine précision choisie), nous sommes parvenus à la convention qu'elle retourne simplement le vecteur nul lui-même.

`Vecteur prod_vect(Vecteur& autre)` effectue le produit vectoriel du `Vecteur` courant et de celui passé en argument. La méthode premièrement vérifie que les deux soient de dimension plus petite ou égale à trois, puis elle fait plonger les deux `Vecteurs` en espace de dimension trois s'ils n'y sont pas déjà et enfin le produit est effectué.

### 3 Classe `ObjetMobile`

La classe `ObjetMobile` sert de base pour introduire des objets physique que l'on veuille dans la suite faire évoluer. Les attributs de la classe sont : `Vecteur pos`, `Vecteur vit` et `Vecteur acc`.

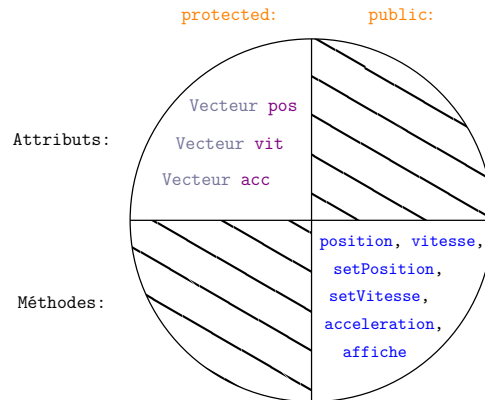


FIGURE 7 – Conception de la classe `ObjetMobile`.

### 4 Classe `ObjetPhysique`

La classe `ObjetPhysique` hérite de la classe `ObjetMobile`, ayant ainsi les attributs de la position, vitesse et de l'accélération. Les `ObjetPhyisques` diffèrent de la classe parente en disposant des attributs `Vecteur champForces`, `ChampForces* champ` et `Contrainte* contrainte`. Les deux derniers sont pointeurs « à la C » qui seront initialisés lors de l'édition des liens dans un `Systeme` en raison qu'il « comprenne » en interne lesquels champs et lesquelles contraintes appliquer à ses objets.

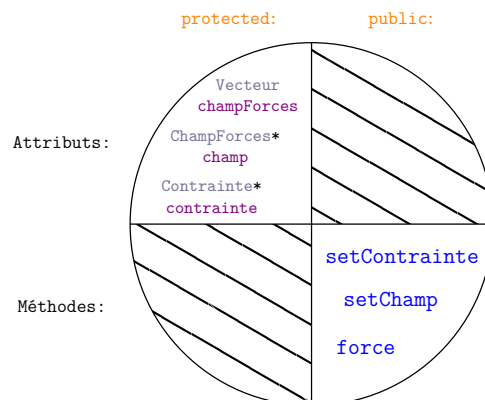


FIGURE 8 – Conception de la classe `ObjetPhysique`.

## 5 Classe `PointMateriel`

Cette classe dispose de deux nouveaux attributs `double masse` et `bool perturbe`. Ce dernier sera bien utile lorsqu'on souhaite poursuivre le `PointMateriel` en question, par exemple afin de compter le nombre de collisions avec d'autres `PointMateriels` ou bien de contacts avec des `Contraintes`.

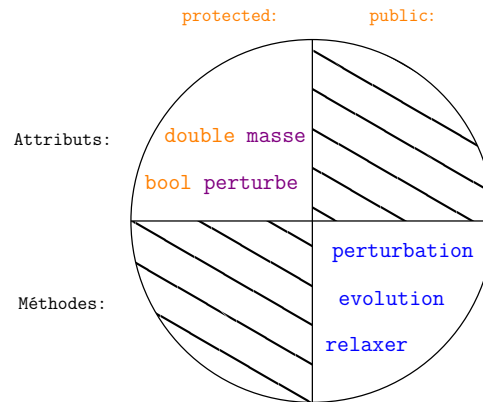


FIGURE 9 – Conception de la classe `PointMateriel`.

Nota bene : on n'introduit `bool perturbe` qu'ici car les `ObjetPhysiques` ne peuvent pas entrer en collisions. De plus, la classe ne dispose pas de constructeur par défaut. Si l'on n'initialise pas un `PointMateriel`, alors sa masse vaut zéro par défaut. Dans ce cas, ce n'est qu'un simple `ObjetPhysique`. De plus, les `PointMateriels` peuvent être des sources des champs, par exemple `ChampNewtonien`, et si la masse est nulle, le champ produit n'est qu'un champ nilpotent. Il vaut mieux éviter de telles complications.

## 6 Classe `PointCharge`

La classe `PointCharge` est une simple extension de `PointMateriel` accordée d'un nouvel attribut `double charge`. Nota bene : la classe ne dispose pas de constructeur par défaut puisque sa classe parente n'en dispose pas non plus.

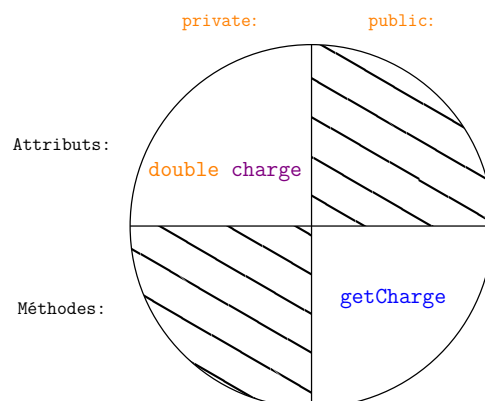


FIGURE 10 – Conception de la classe `Pointcharge`.

## 7 Classes `ChampForces` et `ChampComposee`

Ceux qui agissent sur les `PointMateriels` sont en partie les champs de forces que l'on représente au moyen de la `ChampForces`. Parmi les méthodes publiques y implémentées, celles d'intérêt particulier sont : `force`, qui prend en argument un pointeur « à la C » sur un `PointMateriel` et un `double` temps; `potentiel`, retournant le potentiel sur lequel se trouve un `PointMateriel`; `energiePotentielle`, retournant l'énergie potentielle d'un `PointMateriel`. En raison que la classe `PointMateriel` dispose d'un seul pointeur sur un `ChampForces`, nous avons créé la sous-classe `ChampComposee` dont le seul attribut `std::vector<ChampForces*> champs` permet de juxtaposer plusieurs champs qui puissent agir sur un objet.

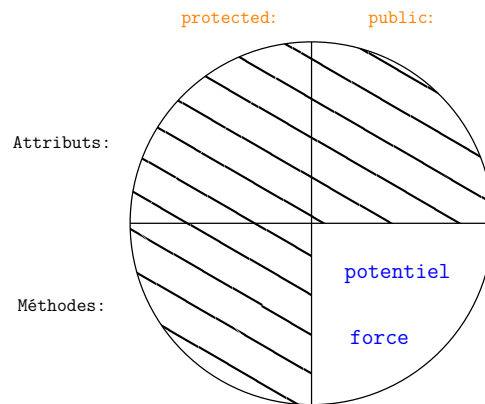
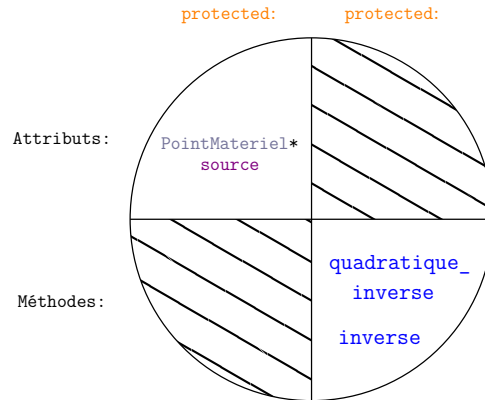


FIGURE 11 – Conception de la classe `ChampForces`.

Nota bene : tandis que les méthodes `force`, `potentiel` et `energiePotentielle` auraient pu être virtuelles pures, on a fait le choix qu'elles retournent simplement le vecteur nul (respectivement zéro) par notre convention. Ceci ne nous pose cependant aucun problème en termes de programmation. Les méthodes revisitées dans `ChampComposee` additionnent simplement les valeurs retournées par chaque `ChampForces*` dans `champ` qui n'est pas un `nullptr`.

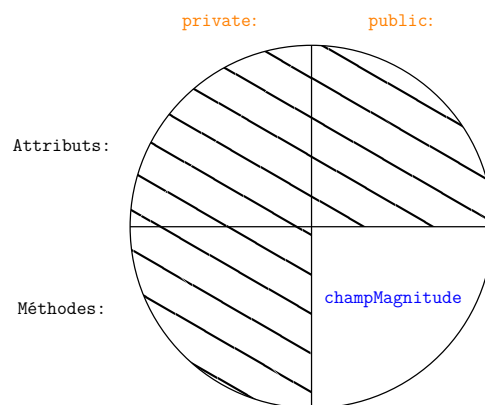
## 8 Classe `ForceCentrale`

Le champ Newtonien et le champ électrostatique d'un `PointCharge` sont les deux des champs de forces centrales. L'unique attribut `source` de `ForceCentrale` est un pointeur « à la C » sur un `PointMateriel`. Il y a deux méthodes protégées : Vecteur `quadratique_inverse` et `double inverse` utilisées par `force` et `potentiel` respectivement. Nota bene : la classe ne dispose pas de constructeur par défaut parce qu'il n'a aucun sens d'avoir la `source` initialisée à `nullptr`. Ainsi, on évite les troubles de vérification si la source est `nullptr` ou non.

FIGURE 12 – Conception de la classe `ForceCentrale`.

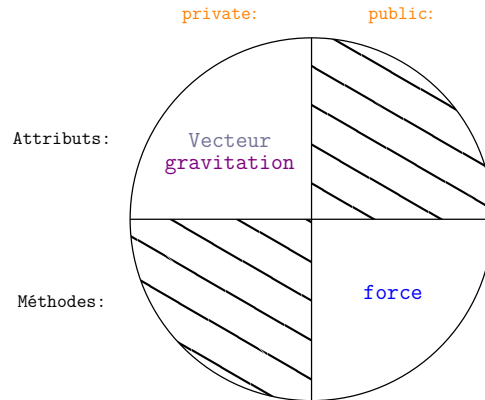
## 9 Classe `ChampNewtonien`

La classe `ChampNewtonien` est notre première implémentation de l'héritage multiple, c'est-à-dire qu'elle hérite de `ChampForces` et de `ForceCentrale`. Les méthodes découlées de `ChampForces` sont revisitées, toutefois avec une nouvelle méthode `champMagnitude`. Nota bene : le constructeur par défaut manque parce que la classe parente `ForceCentrale` n'en dispose pas en raison qu'il faut autrement vérifier dans chaque méthode que la source du champ ne soit pas un `nullptr`.

FIGURE 13 – Conception de la classe `ChampNewtonien`.

## 10 Classe `GravitationConstante`

La classe `GravitationConstante` nous sert à représenter un champ gravitationnel uniforme. On fixe l'unique attribut `Vecteur gravitation` de la classe lors de l'initialisation et celui est multiplié par la masse d'un `PointMateriel` lorsque le champ agit sur ce dernier par la méthode `force`.

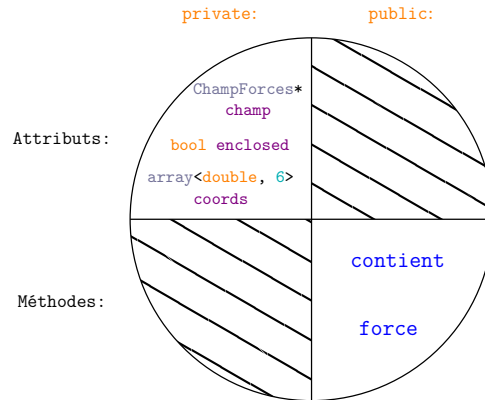
FIGURE 14 – Conception de la classe `GravitationConstante`.

## 11 Classes `ChampElectrique`, `ChampElectriqueConstant` et `ChampMagnetique`

Au monde réel les particules ayant une charge non-nulle créent et interagissent au moyen des champs électromagnétiques. Afin de faire simuler quelques phénomènes électromagnétiques, nous avons implémenté les classes `ChampElectrique`, `ChampElectriqueConstant` et `ChampMagnetique`. La classe `ChampElectrique` hérite à la fois des classes `ChampForces` et `ForceCentrale`, analogue au `ChampNewtonien`, en raison que les `PointCharges` produisent leurs propres champs. Par contre, `ChampElectriqueConstant` est analogue à la classe `GravitationConstante`, et pareillement `ChampMagnetique` est un simple `ChampForces` parce que les monopôles magnétiques n'existent pas. Nota bene : il n'existe pas de constructeur par défaut dans la classe `ChampElectrique` car la classe parente `ForceCentrale` n'en dispose pas.

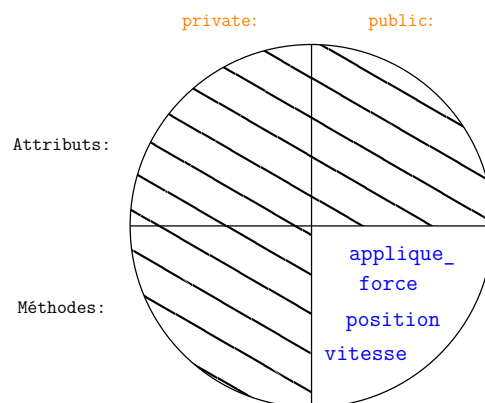
## 12 Classe `ChampRestreint`

Dans certains systèmes physiques, il nécessite de restreindre le volume où un champ puisse exercer son influence. Un exemple en particulier où l'on voulait bien profiter de tels dispositifs, c'est le fonctionnement d'un cyclotron présent dans `exerciceP11-2-cyclotron`. La classe dispose de plusieurs attributs dont : `ChampForces* champ`, un pointeur sur le champ que l'on restreigne, `bool enclosed`, un booléen qui signifie si son domaine d'application se trouve à l'intérieur ou à l'extérieur du volume désigné, `std::array<bool, 6> bornes` qui définit s'il existe des bornes dans une direction spécifique, `std::array<double, 6> coords` qui spécifie les coordonnées des bornes, et `std::array<std::string, 6> bords` qui sert à afficher le `ChampRestreint`. À part la méthode `force` revisitée, la classe dispose de la méthode `contient` qui vérifie si la position du `PointMateriel` reçu en argument réside dans le volume spécifié lors de l'initialisation. Nota bene : la classe ne dispose pas de constructeur par défaut. Sinon, il faut vérifier à chaque pas que `champ` ne soit pas `nullptr`.

FIGURE 15 – Conception de la classe `ChampRestreint`.

## 13 Classes `Contrainte` et `ContrainteComposee`

Les autres agents qui influent les `PointMateriels` sont bien sûr les `Contraintes`. La classe parente dispose de trois méthodes qui modifient (éventuellement) l'évolution d'un `PointMateriel` : `applique_force`, `position` et `vitesse`. Puisque nous sommes parvenus à accorder une seule contrainte à un `PointMateriel`, l'implémentation de la classe `ContrainteComposee` a été nécessaire pour juxtaposer plusieurs `Contraintes`, ainsi la sous-classe dispose de l'unique attribut `std::vector<Contrainte*> contraintes`.

FIGURE 16 – Conception des classes `Contrainte` et `ContrainteComposee`.

La réforme des méthodes dans la classe `ContrainteComposee` qui sont héritées de `Contrainte` est plus délicate que dans sa sous-classe équivalente `ChampComposee`. Tandis que la méthode `applique_force` retourne simplement la somme des valeurs retournées par chaque `Contrainte` dans `contraintes`, le même ne peut pas s'appliquer aux méthodes `position` et `vitesse`. Ainsi, nous sommes parvenus à ce qu'une `ContrainteComposee` itère simplement ces deux méthodes-là sur le `PointMateriel` en question. Nota bene : Les méthodes comme `applique_force` ne sont pas virtuelles pures, mais elles retournent le vecteur nul par notre convention bien que les instances de `Contrainte` ne soient nulle part initialisées. Ceci ne pose aucun problème en termes de programmation ni de conceptualisation à nos yeux.

## 14 Classes Collision, Elastique et Plastique

Pour gérer et exercer les collisions entre deux ou plusieurs `PointMateriels`, nous sommes parvenus à implémenter une sous-classe de `Contrainte` spécifique. Une `Collision` est principalement initialisée par passer en argument un `PointMateriel`, initialisant ainsi l'attribut-pointeur protégé `PointMateriel* autre` sur celui-là, avec lequel d'autres `PointMateriels` peuvent entrer en collision. Une `Collision` dispose aussi d'un attribut privé `double restitution` pour faire simuler des collisions avec une certaine perte d'énergie, et un autre attribut protégé `double tolerance`, un paramètre incontournable en ce qui concerne la détection de collisions. Les deux sous-classes de `Collision` sont `Elastique` dont la restitution est automatiquement initialisé à `1.0`, et `Plastique` dont la restitution vaut `0.0`.

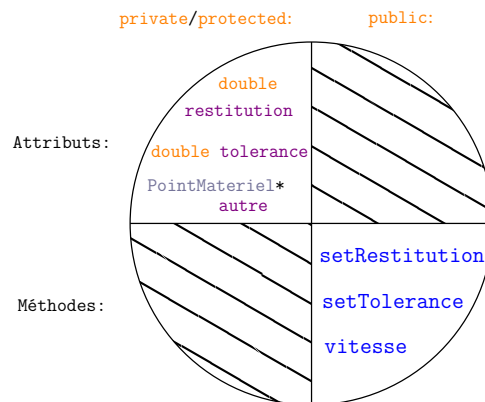


FIGURE 17 – Conception des classes `Collision`, `Elastique` et `Plastique`.

La méthode `vitesse` vérifie en premier lieu si le `PointMateriel* autre` et celui passé en argument sont assez proches pour déclarer qu'une collision est en train d'avoir lieu ou non. Ceci est fait par simplement comparer la distance entre les deux avec la `tolerance` parmi les attributs. Le choix de la valeur de `tolerance` impacte énormément la qualité de détection de collisions. Nous rencontrons ce problème en particulier dans notre simulation « `CompterPi` » où il faut choisir une tolérance assez grande (toutefois pas trop!) pour que les `PointMateriel` qui se déplacent avec une gigantesque vitesse ne s'envolent pas « incorporellement » à travers les autres `PointMateriels`. Ceci arrive en particulier quand il y a un écart géant en terme d'ordre de grandeur entre le pas de temps `dt` et la vitesse courante du `PointMateriel` en question. Le deuxième fait non-trivial que l'on avait remarqué c'est qu'il faut stocker toutes les vitesses retournées par `vitesse`, et seulement après la dernière collision peut-on vraiment mettre à jour les vitesses des objets. Ceci arrive en raison qu'une `Collision` retourne un seul `Vecteur` correspondant à la nouvelle vitesse d'un seul `PointMateriel` et non de l'`autre`. Même si la méthode modifiait les deux objets en même temps, la collision sera détecté par les deux `Collisions` pointant sur les deux objets en question. Le calcul sera ainsi effectué deux fois, ce qui a pour l'effet comme si la collision n'avait même pas eu lieu. Un mot à la fin : la classe ne dispose pas de constructeur par défaut. Dans ce cas, l'attribut `PointMateriel* autre` pointe toujours à un `PointMateriel` existant. Sinon, il faudrait dans chaque méthode vérifier que `autre` ne soit pas un `nullptr`.

## 15 Classe PlanImmobilier

Puisque la plupart des systèmes physiques contient des restrictions spatiales, par exemple un sol ou des murs, nous avons introduit la classe `PlanImmobilier` dont les instances agissent comme des surfaces immobiles de positions et directions quelconques. On initialise un `PlanImmobilier` en définissant son `Vecteur pointReference`, c'est-à-dire un point dans l'espace contenu par le `PlanImmobilier`, et sa `Vecteur normale` qui représente le vecteur normal au plan. Tout `PointMateriel` qui frappe contre le `PlanImmobilier` aura sa vitesse défléchie. Le processus de déflexion est soutenu par les méthodes `position` et `vitesse` revisitées. Le `PlanImmobilier` signale qu'une collision est détectée si un `PointMateriel` se retrouve en dessous du `PlanImmobilier`, ce qui déclenchera par suite les deux méthodes. Le `PointMateriel` aura sa position refixée à la surface du `PlanImmobilier` et sa vitesse déviée. Nota bene : la classe ne dispose pas de constructeur par défaut parce qu'avoir un `PlanImmobilier` non-initialisée n'a aucun sens, en particulier puisque le `Vecteur normal` sera le vecteur nul dans ce cas-là.

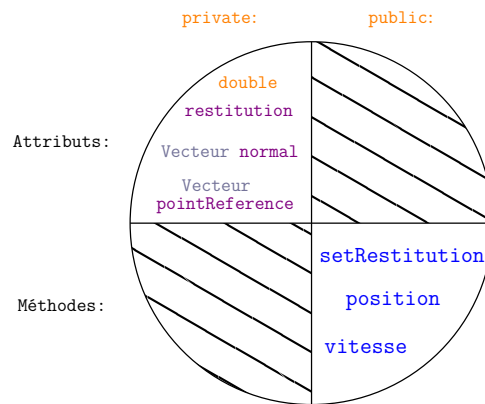
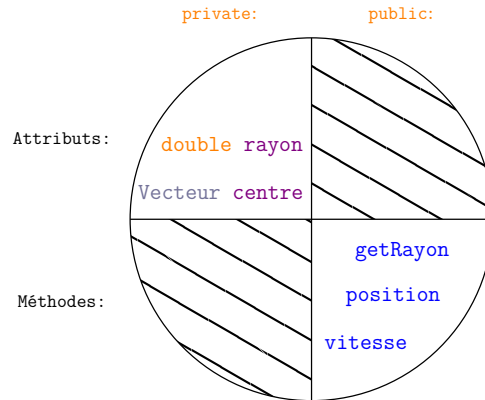


FIGURE 18 – Conception de la classe `PlanImmobilier`.

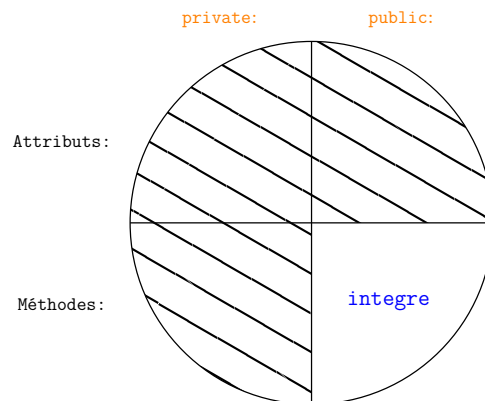
## 16 Classe ContrainteSpherique

Au vu que l'on avait choisi de faire une simulation d'une pendule, il fallait ainsi créer une classe qui restreigne le mouvement de la pendule sur une surface sphérique comme un fil de longueur fixe. La classe `ContrainteSpherique` dispose de deux attributs : le `Vecteur centre` de la surface sphérique représentée et le `double rayon`. Une `ContrainteSpherique` agit sur des `PointMateriels` par voie des méthodes `position` en fixant la position du `PointMateriel` reçu en argument à sa surface, et `vitesse` qui maintient la vitesse toujours tangente à la surface sphérique. Nota bene : la classe ne dispose pas de constructeur par défaut parce qu'il n'a pas de sens d'avoir un rayon nul.

FIGURE 19 – Conception de la classe `ContrainteSpherique`.

## 17 Classes `EulerCromer`, `Newmark` et `RungeKutta`

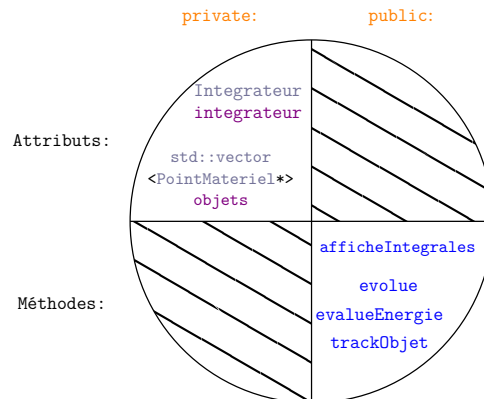
Afin de faire évoluer tout `PointMateriel` dans nos simulations, on a créé une classe parente virtuelle `Integrateur` dont les sous-classes utilisent certains algorithmes d'intégrateurs. La plus simple des sous-classes c'est `EulerCromer`. L'autre intégrateur que l'on avait implémenté utilise l'algorithme de Newmark, et en raison de curiosité, nous sommes pareillement parvenus à introduire l'algorithme de Runge-Kutta. Leur seule méthode est `integre` qui prend en argument un pointeur « à la C » sur un `PointMateriel`, un `double` temps, l'instance lorsqu'on fait évoluer l'objet, et `double` dt.

FIGURE 20 – Conception des classes `EulerCromer`, `Newmark` et `RungeKutta`.

## 18 Classe `Systeme`

Dans nos simulations la collection de `PointMateriels`, `Contraintes` et `ChampForces` est représentée par la classe `Systeme`. De plus, un `Systeme` dispose d'un `double` `t`, l'instant de temps courant, et d'un `Integrateur` `integrateur` qui fait évoluer ses `PointMateriels` selon les `ChampForces` et `Contraintes` leur accordés. Si besoin, on peut poursuivre le mouvement d'un `PointMateriel` en particulier au moyen de l'attribut `int` `tracker` et la méthode `trackObjet` pour que l'on puisse prendre note de ses collisions (qu'on appelle « perturbations ») avec un `unsigned int` `compteur`. Puisque tout vrai système physique est décrit par un système de coordonnées, le nôtre dispose d'un

attribut `Vecteur` `reference` qui sert comme repère du système de coordonnées. Au vu que l'on a choisi de faire simuler le mouvement d'une pendule qui exige une `ContrainteSpherique`, il ne paraît que naturel de fixer le repère au centre de la `ContrainteSpherique` et ainsi afficher les positions des `PointMateriel` en coordonnées sphériques au moyen de la méthode privée `getPositionSpherique`. Pour cette raison, nous sommes parvenus d'introduire les attributs `bool` `spherique`, qui dit s'il existe une `ContrainteSpherique` dans le `Systeme`, et `size_t` `whereSpherique`, qui retient sa position parmi les `std::vector<Contrainte*>` `contraintes`. Par notre convention, on n'aura qu'une seule `ContrainteSpherique` par `Systeme`. On peut relier localement certains `PointMateriels` aux `ChampForces` et/ou `Contraintes` souhaités. Chaque `Systeme` peut évaluer son énergie totale ou l'énergie d'un `PointMateriel` en particulier, et de même pour le moment cinétique. Nota bene : Presque tout constructeur reçoit une liste d'instances passée en argument par référence et non par référence constante. Tandis qu'il est tout à fait possible d'implémenter de tels constructeurs en échangeant les attributs `std::vector<Type*>` en `std::vector<const Type*>`, ceci nous pose beaucoup de problèmes dans presque toutes les méthodes. Bien qu'il paraisse décevant de ne pas inclure le mot-clé, du point de vue programmation, le code est déjà assez « optimisé » (pour nos besoins) en passant les arguments par référence.

FIGURE 21 – Conception de la classe `Systeme`.

## 19 Affichage graphique avec Qt5

L'affichage graphique reprend la structure des classes présentées dans la description des tâches des semaines 8 et 9. Nous avons adapté l'exemple donné pour dessiner tous les éléments physiques via le système qui les contient. De plus, nous avons ajouté les classes complémentaires `GLSphere`, les événements clavier et souris, ainsi que le dessin texturé des cubes. Pour cela, on a adapté des « shaders » afin de pouvoir dessiner à la fois en couleur et avec des images/textures. Pour les `PointMateriels` représentés comme des sphères, nous avons adapté la classe `GLSphere` afin de pouvoir colorer la sphère avec un gradient de couleurs, au lieu d'une couleur de base. Ceci est réalisé en utilisant deux couleurs de base qui se mêlent l'une dans l'autre. De plus, la composition de ces deux couleurs de base résulte d'un tirage aléatoire effectué à l'exécution, pour chaque `PointMateriel` censé d'être représenté par une sphère, en fonction de l'exercice physique simulé.

La partie conceptualisation de comment trouver une approche de dessin conforme aux principes de programmation orientée objet s'est avéré difficile, car nos exercices dessinés constituent un ensemble fini d'instances, de contextes et besoins très différents. Nous avons décidé d'introduire un nouveau fichier header définissant un type de structure appelé `modif` qui contient toutes les caractéristiques

nécessaires au dessin d'un `PointMateriel`, d'une `Contrainte` ou d'un `ChampForces` spécifique (ou bien de leurs sous-classes). Concrètement, on utilise des `doubles` pour faire déplacer, faire pivoter et mettre à l'échelle la `QMatrix4x4` sur laquelle chaque objet est dessiné, ainsi qu'un indicateur renvoyant le type de forme à dessiner (sphère ou cube), même s'il doit être dessiné ou non. Par exemple, dans le programme « Pomme » la contrainte `PlanImmobile` et le `PointMateriel` `terre` définissent la même entité ; elle ne doit ainsi être dessinée qu'une seule fois. De plus, ceux qui nous offrent une meilleure accessibilité, ce sont un type d'`Enum` qui indique l'exercice dessiné, et une balise permettant un meilleur accès aux textures. Le dernier attribut de cette structure est un tableau définissant deux triplets de couleurs RVB utiles pour dessiner les gradients des sphères. Ce dernier type, `modif`, est utilisé comme attribut dans les classes `PointMateriel`, `Contrainte` et `ChampForces`, et est initialisé par défaut avec les valeurs de base pour éviter de passer par un constructeur. On dispose des méthodes « set » et « get » pour définir, à la rigueur, ces valeurs individuellement en fonction de la simulation, et accéder directement aux paramètres dans les méthodes `dessine` de `VueOpenGL`.

Un problème similaire de mauvaise modularisation s'est également posé concernant la gestion des textures. Nous avons dû distinguer les cas où un cube devait être dessiné avec une texture ou une simple couleur suffisait, ainsi que déterminer précisément quelle texture appliquer à chaque face du cube. Pour cela, nous avons introduit un attribut `map` de type `std::map<Exercice, std::vector<std::vector<std::unique_ptr<QOpenGLTexture>>>>` dans la classe `VueOpenGL`, initialisé pour chaque exercice avec une matrice de textures `QOpenGLTexture`. Cette approche générale prévoit l'utilisation d'une texture pour les côtés latéraux du cube, une pour sa face supérieure, et une autre pour sa face inférieure. Ainsi, selon le type de cube à dessiner, il est possible d'accéder directement à la texture souhaitée dans la méthode `dessineCubeTexture` de `VueOpenGL`.

Nota bene : cette approche reste imparfaite dans sa généralisation. En effet, le programmeur des méthodes graphiques doit encore, dans certains cas, prendre des décisions spécifiques à chaque exercice concernant ce qui doit être dessiné et de quelle manière. Cela nous a conduit à ajouter plusieurs attributs aux classes à dessiner, afin d'accéder à des informations auparavant inaccessibles aux méthodes `dessine`. Nous sommes bien au courant que dans un contexte plus professionnel chaque exercice aurait mérité d'être davantage spécialisé, avec ses propres classes dérivées et les méthodes de dessin spécifiques.